1. Git Basics

What is Git?

Git is a distributed version control system (VCS) that is widely used for tracking changes in source code during software development. It was created by Linus Torvalds in 2005 and has since become the de facto standard for version control in the software development industry.

Git allows multiple developers to collaborate on a project by providing a history of changes, facilitating the tracking of who made what changes and when. Here are some key concepts and features of Git:

1. **Repository (Repo):** A Git repository is a directory or storage location where your project's files and version history are stored. There can be a local repository on your computer and remote repositories on servers.
2. **Commits:** In Git, a commit is a snapshot of your project at a particular point in time. Each commit includes a unique identifier, a message describing the changes, and a reference to the previous commit.
3. **Branches:** Branches in Git allow you to work on different features or parts of your project simultaneously without affecting the main development line (usually called the "master" branch). Branches make it easy to experiment, develop new features, and merge changes back into the main branch when they are ready.
4. **Pull Requests (PRs):** In Git-based collaboration workflows, such as GitHub or GitLab, pull requests are a way for developers to propose changes and have them reviewed by their peers. This is a common practice for open-source and team-based projects.
5. **Merging:** Merging involves combining changes from one branch (or multiple branches) into another. When a branch's changes are ready to be incorporated into the main branch, you can merge them.
6. **Remote Repositories:** Remote repositories are copies of your project stored on a different server. Developers can collaborate by pushing their changes to a remote repository and pulling changes from it. Common remote repository hosting services include GitHub, GitLab, and Bitbucket.
7. **Cloning:** Cloning is the process of creating a copy of a remote repository on your local machine. This allows you to work on the project and make changes locally.
8. **Forking:** Forking is a way to create your copy of a repository, typically on a hosting platform like GitHub. You can make changes to your fork without affecting the original project and later create pull requests to contribute your changes back to the original repository.

Git is known for its efficiency, flexibility, and ability to handle both small and large-scale software projects. It is used not only for software development but also for managing and tracking changes in various types of text-based files, including documentation and configuration files. Learning Git is essential for modern software development and collaboration.

Why we need git?

Git is an essential tool in software development and for many other collaborative and version-controlled tasks. Here are some key reasons why Git is crucial:

1. **Version Control:** Git allows you to track changes in your project's files over time. It provides a complete history of all changes, making it easy to understand what was done, when it was done, and who made the changes. This is invaluable for debugging, auditing, and collaboration.
2. **Collaboration:** Git enables multiple developers to work on the same project simultaneously without interfering with each other's work. It provides mechanisms for merging changes made by different contributors and resolving conflicts when they occur.
3. **Branching:** Git supports branching, which allows developers to create isolated environments for developing new features or fixing bugs. This is essential for managing complex software projects and experimenting with new ideas without affecting the main codebase.
4. **Distributed Development:** Git is a distributed version control system, meaning that every developer has a complete copy of the project's history on their local machine. This provides redundancy, facilitates offline work, and reduces the reliance on a central server.
5. **Backup and Recovery:** With Git, your project's history is distributed across multiple locations, including local and remote repositories. This provides redundancy and makes it easy to recover from accidental data loss or system failures.
6. **Code Review:** Git-based platforms like GitHub, GitLab, and Bitbucket provide tools for code review and collaboration. Developers can propose changes, comment on code, and discuss improvements, making it easier to maintain code quality.
7. **Open Source and Community Development:** Git has become the standard for open-source software development. It allows anyone to fork a project, make contributions, and create pull requests, which makes it easy for communities of developers to collaborate on a single codebase.
8. **Efficiency:** Git is designed to be fast and efficient. It only stores the changes made to files, rather than entire file copies, which results in small repository sizes and faster operations.
9. **History and Documentation:** Git's commit history and commit messages serve as a form of documentation. It's easier to understand the context and reasoning behind a change by looking at the commit history and associated messages.
10. **Customizability:** Git is highly configurable and extensible. You can set up hooks and scripts to automate workflows, enforce coding standards, and integrate with various tools.

In summary, Git is essential for tracking changes in your projects, facilitating collaboration among developers, and ensuring the integrity and version history of your code. Whether you're working on a personal project or as part of a large team, Git is a fundamental tool for modern software development and version control.

What is Version Control System (VCS)?

A Version Control System (VCS), also commonly referred to as a Source Code Management (SCM) system, is a software tool or system that helps manage and track changes to files and directories over time. The primary purpose of a VCS is to keep a historical record of all changes made to a set of files, allowing multiple people to collaborate on a project while maintaining the integrity of the codebase. There are two main types of VCS: centralized and distributed.

Centralized Version Control Systems (CVCS): In a CVCS, there is a single central repository that stores all the project files and their version history. Developers check out files from this central repository, make changes, and then commit those changes back to the central repository. Examples of CVCS include CVS (Concurrent Versions System) and Subversion (SVN).

Distributed Version Control Systems (DVCS): In a DVCS, every developer has a complete copy of the project's repository, including its full history, on their local machine. This allows developers to work independently, create branches for experimentation, and synchronize their changes with remote repositories. Git is the most well-known and widely used DVCS, but other DVCS options include Mercurial and Bazaar.

Key features and benefits of Version Control Systems include:

1. **History Tracking:** VCS systems maintain a complete history of changes, including who made the change, what was changed, and when it was changed. This makes it easy to review and understand the evolution of a project.
2. **Collaboration:** VCS allows multiple developers to work on the same project simultaneously. It provides mechanisms for merging changes made by different contributors and resolving conflicts when they occur.
3. **Branching and Isolation:** VCS systems support branching, allowing developers to create isolated environments for new features or bug fixes. This isolates changes and helps manage complex development tasks.
4. **Revert and Rollback:** If a mistake is made, it is possible to revert changes to a previous state or commit. This is essential for error correction and maintaining code quality.
5. **Backup and Recovery:** Project data is stored in multiple locations, providing redundancy and facilitating data recovery in case of accidental data loss or system failures.
6. **Documentation:** Commit messages and history serve as a form of documentation, explaining why a change was made, who made it, and when it was made.
7. **Efficiency:** VCS systems are designed to be fast and efficient. They typically store only the changes made to files, rather than entire file copies, which results in small repository sizes and faster operations.

VCS is a fundamental tool in software development and is used not only for source code but also for tracking changes in documentation, configuration files, and other types of text-based files. It is especially crucial for collaborative projects, allowing teams of developers to work together on the same codebase with confidence.

Git Life Cycle

The Git lifecycle refers to the typical sequence of actions and steps you take when using Git to manage your source code and collaborate with others. Here's an overview of the Git lifecycle:

1. **Initializing a Repository**:
   - To start using Git, you typically initialize a new repository (or repo) in your project directory. This is done with the command git init.
2. **Working Directory**:
   - Your project files exist in the working directory. These are the files you are actively working on.
3. **Staging**:
   - Before you commit changes, you need to stage them. Staging allows you to select which changes you want to include in the next commit. You use the git add command to stage changes selectively or all at once with git add ..
4. **Committing**:
   - After you've staged your changes, you commit them with a message explaining what you've done. Commits create snapshots of your project at that point in time. You use the git commit command to make commits, like git commit -m "Add new feature".
5. **Local Repository**:
   - Commits are stored in your local repository. Your project's version history is preserved there.
6. **Branching**:
   - Git encourages branching for development. You can create branches to work on new features, bug fixes, or experiments without affecting the main codebase. Use the git branch and git checkout commands for branching.
7. **Merging**:
   - After you've completed work in a branch and want to integrate it into the main codebase, you perform a merge. Merging combines the changes from one branch into another. Use the git merge command.
8. **Remote Repository**:
   - For collaboration, you can work with remote repositories hosted on servers like GitHub, GitLab, or Bitbucket. These repositories serve as a central hub for sharing code.
9. **Pushing**:
   - To share your local commits with a remote repository, you push them using the git push command. This updates the remote repository with your changes.
10. **Pulling**:
    - To get changes made by others in the remote repository, you pull them to your local repository with the git pull command. This ensures that your local copy is up to date.

11. **Conflict Resolution**:
    - Conflicts can occur when multiple people make changes to the same part of a file. Git will inform you of conflicts, and you must resolve them by editing the affected files manually.
12. **Collaboration**:
    - Developers can collaborate by pushing, pulling, and making pull requests in a shared remote repository. Collaboration tools like pull requests are commonly used on platforms like GitHub and GitLab.
13. **Tagging and Releases**:
    - You can create tags to mark specific points in the project's history, such as version releases. Tags are useful for identifying significant milestones.
14. **Continuous Cycle**:
    - The Git lifecycle continues as you repeat these steps over time to manage the ongoing development and evolution of your project. This cycle supports collaborative and agile software development.

The Git lifecycle allows for effective version control, collaboration, and the management of complex software projects. It provides a structured approach to tracking and sharing changes, enabling multiple developers to work together on a project with minimal conflicts and a clear history of changes.

2. Git Installation

To install Git on your computer, you can follow the steps for your specific operating system:

1. Installing Git on Windows:

a. **Using Git for Windows (Git Bash):**

- Go to the official Git for Windows website: https://gitforwindows.org/
- Download the latest version of Git for Windows.
- Run the installer and follow the installation steps. You can choose the default settings for most options.

b. Using GitHub Desktop (Optional):

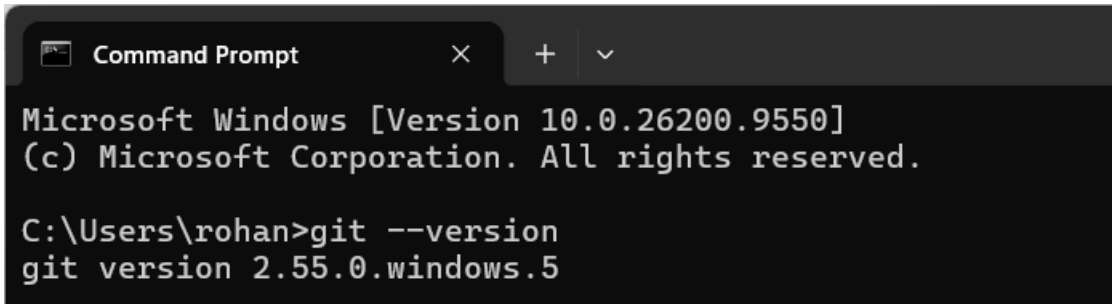
- If you prefer a graphical user interface (GUI) for Git, you can also install GitHub Desktop, which includes Git. Download it from https://desktop.github.com/ and follow the installation instructions.

2. Installing Git from Source (Advanced):

- If you prefer to compile Git from source, you can download the source code from the official Git website (https://git-scm.com/downloads) and follow the compilation instructions provided there. This is usually only necessary for advanced users.

1. After installation, you can open a **terminal or command prompt and verify that Git is correctly installed by running the following command:**

$ git –version



If Git is installed successfully, you will see the Git version displayed in the terminal. You can now start using Git for version control and collaborate on software development projects.

How to Configure the Git?

Configuring Git involves setting up your identity (your name and email), customizing Git options, and configuring your remote repositories. Git has three levels of configuration: system,

global, and repository-specific. Here's how you can configure Git at each level:

1. System Configuration:

- System-level configuration affects all users on your computer. It is typically used for site-specific configurations and is stored in the /etc/gitconfig file.

userTo set system-level configuration, you can use the git config command with the --system flag (usually requires administrator privileges). For example:

$ git config --system user.name "Your Name"
$ git config --system user.email "your.email@example.com"

1. Global Configuration:

- Global configuration is specific to your user account and applies to all Git repositories on your computer. This is where you usually set your name and email.

To set global configuration, you can use the git config command with the --global flag.

A. Set username & email:

For example:

$ git config --global user.name "Your Name"
$ git config --global user.email "your.email@example.com"

```
Dell@DESKTOP-27414RD MINGW64 ~
$ git config --global user.name ptv

Dell@DESKTOP-27414RD MINGW64 ~
$ git config --global user.email "priyanka87pihu@gmail.com"
```

You can also view your global Git configuration by using:

$ git config --global --list

```
Dell@DESKTOP-27414RD MINGW64 ~
$ git config --global --list
user.name=ptv
user.email=priyanka87pihu@gmail.com
```

Check whether it worked:

$ git config --global user.name

```
Dell@DESKTOP-27414RD MINGW64 ~ (master)
$ git config --global user.name
ptv
```

$ git config --global user.email

```
Dell@DESKTOP-27414RD MINGW64 ~ (master)
$ git config --global user.email
priyanka87pihu@gmail.com
```

B. Git Commands List

Git is a popular version control system used for tracking changes in software development projects. Here's a list of common Git commands along with brief explanations:

Check your current directory - Print working directory:

```
Dell@DESKTOP-27414RD MINGW64 ~ (master)
$ pwd
/c/Users/Dell
```

To initialize GIT repository where empty container is created:

```
rohan@JAHOVAH MINGW64 ~
$ git init
Initialized empty Git repository in C:/Users/rohan/.git/
```

2. Create a new directory:

Use the `mkdir` command. (demo newly dreated new directory)

mkdir mytest

```
rohan@JAHOVAH MINGW64 ~ (master)
$ mkdir mytest
```

To check:

ls

```
Dell@DESKTOP-27414RD MINGW64 ~ (master)
$ ls
 AppData/
'Application Data'@
 Contacts/
 Cookies@
 Desktop/
 Documents/
 Downloads/
 Favorites/
 IntelGraphicsProfiles/
 Links/
'Local Settings'@
 Music/
'My Documents'@
 NTUSER.DAT
 NTUSER.DAT{2ad838bc-efea-11ee-a54d-000d3a94eaa1}.TM.blf
 NTUSER.DAT{2ad838bc-efea-11ee-a54d-000d3a94eaa1}.TMContainer00000000000000000001.regtra
ns-ms
 NTUSER.DAT{2ad838bc-efea-11ee-a54d-000d3a94eaa1}.TMContainer00000000000000000002.regtra
ns-ms
 NetHood@
 OneDrive/
 Pictures/
 PrintHood@
 Recent@
'Saved Games'/
 Searches/
 SendTo@
'Start Menu'@
 Templates@
 Videos/
'VirtualBox VMs'/
'WPS Cloud'/
 miktex-console.lock
 myproject/
 ntuser.dat.LOG1
 ntuser.dat.LOG2
 ntuser.ini
```

3. Move into the directory(Change directory)

cd mytest

```
rohan@JAHOVAH MINGW64 ~ (master)
$ cd mytest

rohan@JAHOVAH MINGW64 ~/mytest (master)
```

to check:

pwd

```
rohan@JAHOVAH MINGW64 ~/mytest (master)
$ pwd
/c/Users/rohan/mytest
```

4. Create git

git init: Initializes a new Git repository in the current directory.(to run)

(**master** – already initialized)

```
rohan@JAHOVAH MINGW64 ~/mytest (master)
$ git init
Initialized empty Git repository in C:/Users/rohan/mytest/.git/
```

Command	Meaning
`pwd`	Shows where you currently are
`mkdir mytest`	Creates a new folder
`cd mytest`	Enters the folder
`git init`	Creates a Git repository in that folder

Important: `git init` does **not** create the project folder. You create the folder first using `mkdir`, enter it using `cd`, and then use `git init`.

1. **git clone <repository URL>**: Creates a copy of a remote repository on your local machine.
2. **git add <file>**: Stages a file to be committed, marking it for tracking in the next commit.
3. **git commit -m "message"**: Records the changes you've staged with a descriptive commit message.
4. **git status**: Shows the status of your working directory and the files that have been modified or staged.
5. **git log**: Displays a log of all previous commits, including commit hashes, authors, dates, and commit messages.
6. **git diff**: Shows the differences between the working directory and the last committed version.
7. **git branch**: Lists all branches in the repository and highlights the currently checked-out branch.
8. **git branch <branchname>**: Creates a new branch with the specified name.
9. **git checkout <branchname>**: Switches to a different branch.
10. **git merge <branchname>**: Merges changes from the specified branch into the currently checked-out branch.
11. **git pull**: Fetches changes from a remote repository and merges them into the current branch.
12. **git push**: Pushes your local commits to a remote repository.
13. **git remote**: Lists the remote repositories that your local repository is connected to.
14. **git fetch**: Retrieves changes from a remote repository without merging them.
15. **git reset <file>**: Unstages a file that was previously staged for commit.
16. **git reset --hard <commit>**: Resets the branch to a specific commit, discarding all changes after that commit.
17. **git stash**: Temporarily saves your changes to a "stash" so you can switch branches without committing or losing your work.
18. **git tag**: Lists and manages tags (usually used for marking specific points in history, like releases).
19. **git blame <file>**: Shows who made each change to a file and when.
20. **git rm <file>**: Removes a file from both your working directory and the Git repository.
21. **git mv <oldfile> <newfile>**: Renames a file and stages the change.

These are some of the most common Git commands, but Git offers a wide range of features and

options for more advanced usage. You can use git --help followed by the command name to get more information about any specific command, e.g., git help commit.

Experiments On

Project Management with Git

Experiment 1: Setting Up and Basic Commands:

Initialize a new Git repository in a directory. Create a new file and add it to the staging area and commit the changes with an appropriate commit message.

Solution:

To initialize a new Git repository in a directory, create a new file, add it to the staging area, and commit the changes with an appropriate commit message, follow these steps:

1. Open your terminal and navigate to the directory where you want to create the Git repository.
2. **Initialize a new Git repository in that directory:**

$ git init

```
rohan@JAHOVAH MINGW64 ~/mytest (master)
$ git init
Initialized empty Git repository in C:/Users/rohan/mytest/.git/
```

3. Create a new file in the directory. Create a new file using echo …For example, let's create a file named "my_file.txt." You can use any text editor or command-line tools to create the file.

   $echo my first ever test file >file.txt

```
rohan@JAHOVAH MINGW64 ~/mytest (master)
$ echo my first ever test file > file.txt
```

   To verify:

   $ls

```
rohan@JAHOVAH MINGW64 ~/mytest (master)
$ ls
file.txt
```

4. Add the newly created file to the staging area. Replace "my_file.txt" with the actual name

of your file:

Add the file to the staging area:

1. Add all file : $ git add .

```
rohan@JAHOVAH MINGW64 ~/mytest (master)
$ git add .
warning: in the working copy of 'file.txt', LF will be replaced by CRLF the next time Git touches it
```

2. Add specific file : $ git add my_file.txt

```
rohan@JAHOVAH MINGW64 ~/mytest (master)
$ git add file.txt
```

Add other file:

```
rohan@JAHOVAH MINGW64 ~/mytest (master)
$ echo second test file > hello.txt

rohan@JAHOVAH MINGW64 ~/mytest (master)
$ echo third test file > welcome.txt
```

```
rohan@JAHOVAH MINGW64 ~/mytest (master)
$ git add hello.txt
warning: in the working copy of 'hello.txt', LF will be

rohan@JAHOVAH MINGW64 ~/mytest (master)
$ git add welcome.txt
warning: in the working copy of 'welcome.txt', LF will b
```

This command stages the file for the upcoming commit.

5. Commit the changes with an appropriate commit message. Replace "Your commit message here" with a meaningful description of your changes:

   $ git commit -m mytest

```
rohan@JAHOVAH MINGW64 ~/mytest (master)
$ git commit -m mytest
[master (root-commit) 7f214e0] mytest
 3 files changed, 3 insertions(+)
 create mode 100644 file.txt
 create mode 100644 hello.txt
 create mode 100644 welcome.txt
```

Your commit message should briefly describe the purpose or nature of the changes you made. For example:

To check status of commit: $git status

```
rohan@JAHOVAH MINGW64 ~/mytest (master)
$ git status
On branch master
nothing to commit, working tree clean
```

After these steps, your changes will be committed to the Git repository with the provided commit message. You now have a version of the repository with the new file and its history stored in Git.

Experiment 2: Creating and Managing Branches:

Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "feature-branch" into "master." {CURD OPERATIONS}

Solution:

1. Make sure you are in the **"master" branch** by switching to it:

$ git checkout master

```
rohan@JAHOVAH MINGW64 ~/sample (master)
$ git checkout master
Already on 'master'
```

To create a new branch named "feature-branch," switch to the "master" branch, and merge the "feature-branch" into "master" in Git, follow these steps:

Step 1: Check the current branch

```
rohan@JAHOVAH MINGW64 ~/sample (master)
$ git branch
* master
```

Step 2: Create feature-branch

```
rohan@JAHOVAH MINGW64 ~/sample (master)
$ git branch feature-branch
```

Check the branches:

```
rohan@JAHOVAH MINGW64 ~/sample (master)
$ git branch
  feature-branch
* master
```

The branch is created, but you are **still on master**.

Step 3: Switch to feature-branch

```
rohan@JAHOVAH MINGW64 ~/sample (master)
$ git checkout feature-branch
Switched to branch 'feature-branch'
```

OR

```
rohan@JAHOVAH MINGW64 ~/sample (feature-branch)
$ git switch feature-branch
Already on 'feature-branch'
```

Check: $git branch

CURD Operations:

C — Create : Create a new file:

```
rohan@JAHOVAH MINGW64 ~/sample (feature-branch)
$ echo "this is a feature" > feature.txt
```

To check: ls

Then add and commit:

```
rohan@JAHOVAH MINGW64 ~/sample (feature-branch)
$ git add feature.txt
warning: in the working copy of 'feature.txt', LF will be replaced by CRLF the next time Git touches it

rohan@JAHOVAH MINGW64 ~/sample (feature-branch)
$ git commit -m "create feature file"
[feature-branch 9235de9] create feature file
 1 file changed, 1 insertion(+)
 create mode 100644 feature.txt
```

R — Read : Display the file:

```
rohan@JAHOVAH MINGW64 ~/sample (feature-branch)
$ cat feature.txt
this is a feature
```

U — Update : Add another line:

```
rohan@JAHOVAH MINGW64 ~/sample (feature-branch)
$ echo "feature branch content updated" >> feature.txt
```

To check:

```
rohan@JAHOVAH MINGW64 ~/sample (feature-branch)
$ cat feature.txt
this is a feature
feature branch content updated
```

Then add and commit:

```
rohan@JAHOVAH MINGW64 ~/sample (feature-branch)
$ git add feature.txt
warning: in the working copy of 'feature.txt', LF

rohan@JAHOVAH MINGW64 ~/sample (feature-branch)
$ git commit -m "update feature file"
[feature-branch 20d955e] update feature file
 1 file changed, 1 insertion(+)
```

D — Delete (don't use this option, since we have to merge later)

rm feature.txt
git add -u
git commit -m "Delete feature file"

Step 4: Switch back to master

```
rohan@JAHOVAH MINGW64 ~/sample (feature-branch)
$ git checkout master
Switched to branch 'master'
```

To check :

```
rohan@JAHOVAH MINGW64 ~/sample (master)
$ git branch
  feature-branch
* master
```

Step 5: Merge feature-branch into master

```
rohan@JAHOVAH MINGW64 ~/sample (master)
$ git merge feature-branch
Updating cfbdb15..20d955e
Fast-forward
 feature.txt | 2 ++
 1 file changed, 2 insertions(+)
 create mode 100644 feature.txt
```

Git will merge the changes from feature-branch into master.

Step 6: Check the result

```
rohan@JAHOVAH MINGW64 ~/sample (master)
$ git log --oneline --all
20d955e (HEAD -> master, feature-branch) update feature file
9235de9 create feature file
cfbdb15 sample
0c976df sample

rohan@JAHOVAH MINGW64 ~/sample (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Experiment 3: Creating and Managing Branches:

Write the commands to stash your changes, switch branches, and then apply the stashed changes.

Aim:

To learn how to temporarily save changes, switch to another branch, and reapply the saved changes using Git

Step 1: Stash Your Changes

- Saves your uncommitted changes temporarily.
- Cleans your working directory so you can switch branches.
- Your changes are stored in the Git stash.

Example: Suppose you modify index.html but are not ready to commit the changes. You can use git stash to save them temporarily.

$ git stash

Step 2: Switch to Another Branch

- Switches from your current branch to another branch.
- Replace branch-name with the actual branch name.

$ git switch branch-name

Example:

$ git switch main

This switches to the main branch.

{{{{{{ **Note: If the branch does not exist, you can create and switch to it using:**

$ git switch -c new-branch }}}}}}

Step 3: Apply the Stashed Changes

- Restores the previously stashed changes to your working directory.
- Applies the changes to the branch you are currently on.
- Keeps the saved changes in the stash list, allowing you to apply them again if needed.

$ git stash apply

Step 4: Check the Stash List (Optional)

- Displays all changes saved in the stash.
- Helps you check whether your changes are still stored.

$ git stash list

```
rohan@JAHOVAH MINGW64 ~/demo (master)
$ git switch - c feature
fatal: only one reference expected

rohan@JAHOVAH MINGW64 ~/demo (master)
$ git switch -c feature
fatal: a branch named 'feature' already exists

rohan@JAHOVAH MINGW64 ~/demo (master)
$ git switch -c features
Switched to a new branch 'features'

rohan@JAHOVAH MINGW64 ~/demo (features)
$ echo "hello git" > index.html

rohan@JAHOVAH MINGW64 ~/demo (features)
$ git add index.html
warning: in the working copy of 'index.html', LF will be replaced by CRLF the next

rohan@JAHOVAH MINGW64 ~/demo (features)
$ git commit -m "add index.html"
[features a434451] add index.html
 3 files changed, 4 insertions(+)
 create mode 100644 index.html
 create mode 100644 new.txt
 create mode 100644 try.txt

rohan@JAHOVAH MINGW64 ~/demo (features)
$ git stash
No local changes to save

rohan@JAHOVAH MINGW64 ~/demo (features)
$ git switch master
Switched to branch 'master'

rohan@JAHOVAH MINGW64 ~/demo (master)
$ git stash apply
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   new.txt
        new file:   try.txt


rohan@JAHOVAH MINGW64 ~/demo (master)
$ git stash list
stash@{0}: WIP on master: 96c35ff update feature file
```

1. Create and switch to a new branch
$git switch -c features

2. Create a file and commit it
$echo "Hello Git" > index.html
$git add index.html
$git commit -m "Add index.html"

3. Modify the file
$echo "Learning Git stash" >> index.html

4. Stash the changes
$git stash

5. Switch to another branch
$git switch master

6. Apply the stashed changes
$git stash apply

7. Check the stash list
$ git stash list

Result:

Successfully learned how to stash uncommitted changes, switch branches, and apply the stashed changes using Git.

Important note :

git stash apply does not remove the stash entry. If you want to apply the changes and remove the stash entry after a successful application, use git stash pop instead.

Experiment 4: Collaboration and Remote Repositories

Clone a remote Git repository to your local machine.

Aim:
To clone a remote Git repository to a local machine using Git.

Step 1: Copy the Repository URL

Open the repository on GitHub and click **Code**. Copy the repository URL.

https://github.com/username/repository.git

Step 2: Clone the Repository

Open your terminal or Git Bash and execute:

- git clone downloads a copy of the remote repository to your local machine.
- The URL specifies the remote repository to clone.
- Git downloads the project files, commit history, and branches.

$ git clone https://github.com/username/repository.git

Step 3: Navigate to the Repository

- cd stands for *Change Directory*.
- It moves you into the cloned repository folder.

$ cd repository

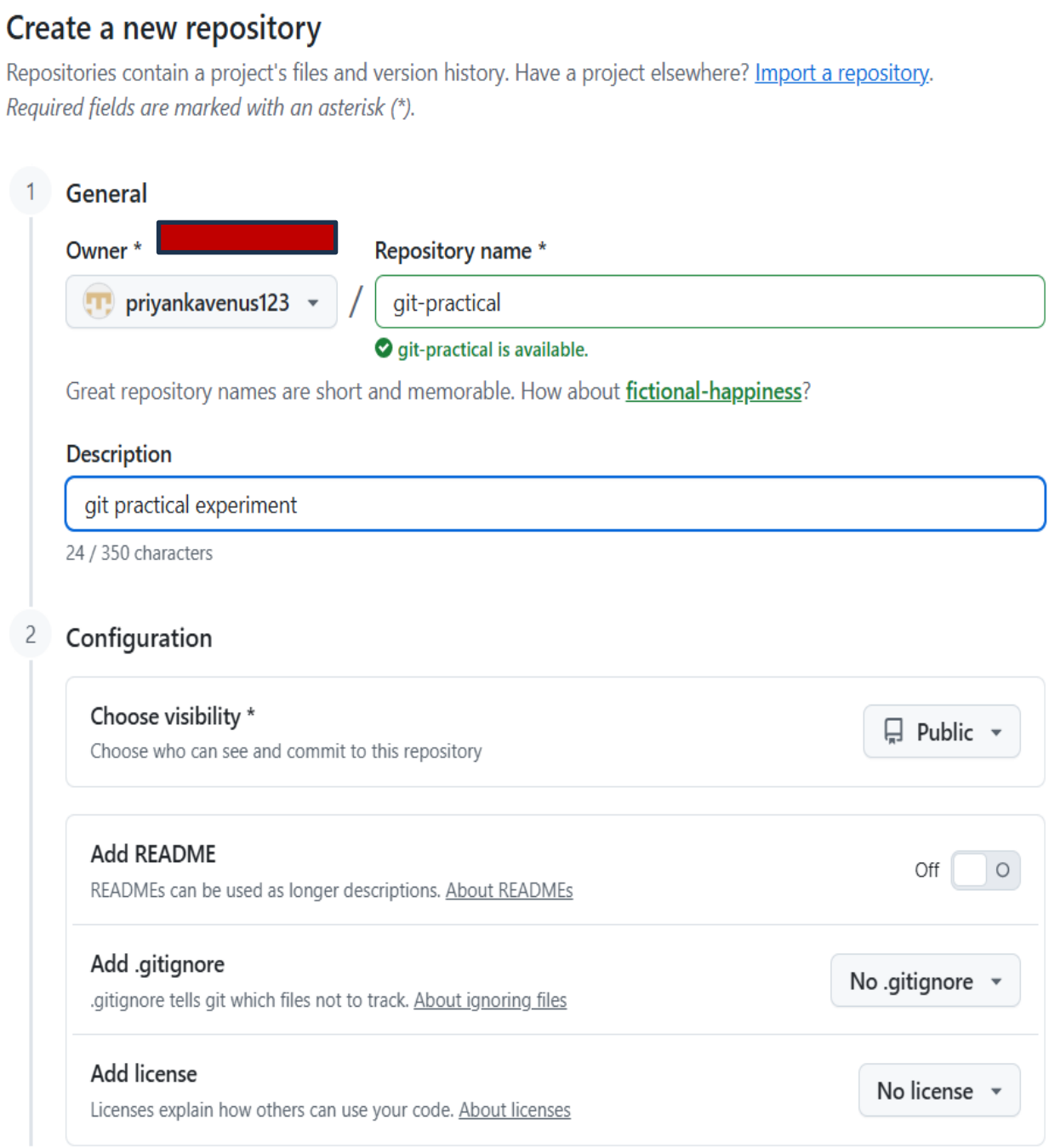
Step 4: Verify the Remote Repository

- Displays the remote repository URLs.
- By default, the remote repository is named origin.
- It shows the URLs used for fetching and pushing changes.

  $ git remote -v

Prework and steps:

1. Visit : https://github.com/
2. Signup(if no count create)/ signin(if account is exists)
3. Login
4. Create a New Repository : Click the + icon in the upper-right corner → Select **New repository**.
5. Fill in the Repository Details



Code → copy URL → https://github.com/priyankavenus123/git-practical

6. Clone the Repository Using Git Bash : Open Git Bash and enter: cd ~/desktop and enter

```
rohan@JAHOVAH MINGW64 ~/demo (master)
$ cd ~/Desktop

rohan@JAHOVAH MINGW64 ~/Desktop (master)
$ git clone https://github.com/priyankavenus123/git-practical
Cloning into 'git-practical'...
warning: You appear to have cloned an empty repository.
```

7. Enter the Repository

```
rohan@JAHOVAH MINGW64 ~/Desktop (master)
$ cd git-practical

rohan@JAHOVAH MINGW64 ~/Desktop/git-practical (main)
$ git remote -v
origin  https://github.com/priyankavenus123/git-practical (fetch)
origin  https://github.com/priyankavenus123/git-practical (push)
```

Local repository is now connected to GitHub.

Result:

Successfully cloned a remote Git repository to the local machine and verified the remote repository connection.